In [2]:
```
import pandas as pd
```

In [5]:
```
data = pd.read_csv(r"C:\Users\yashw\Downloads\mini_project.csv")

```

In [6]:
```
data.head()
```

Out[6]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance
0	1	15634602	Hargrave	619	France	Female	42	2	0.00
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86
2	3	15619304	Onio	502	France	Female	42	8	159660.80
3	4	15701354	Boni	699	France	Female	39	1	0.00
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82

In [7]:
```
data.tail()
```

Out[7]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Bala
9995	9996	15606229	Obijiaku	771	France	Male	39	5	
9996	9997	15569892	Johnstone	516	France	Male	35	10	5736
9997	9998	15584532	Liu	709	France	Female	36	7	
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3	7507
9999	10000	15628319	Walker	792	France	Female	28	4	13014

In [8]:
```
data.shape
```

Out[8]:
```
(10000, 14)
```

In [9]:
```
print("Number of Rows",data.shape[0])
print("Number of Columns",data.shape[1])
```

```
Number of Rows 10000
Number of Columns 14
```

In [10]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

In [11]:
```
data.isnull().sum()
```

Out[11]:
```
RowNumber          0
CustomerId         0
Surname            0
CreditScore        0
Geography          0
Gender             0
Age                0
Tenure             0
Balance            0
NumOfProducts      0
HasCrCard          0
IsActiveMember     0
EstimatedSalary    0
Exited             0
dtype: int64
```

In [12]:
```
data.describe(include='all')
```

Out[12]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age
count	10000.00000	1.000000e+04	10000	10000.000000	10000	10000	10000.000000
unique	NaN	NaN	2932	NaN	3	2	NaN
top	NaN	NaN	Smith	NaN	France	Male	NaN
freq	NaN	NaN	32	NaN	5014	5457	NaN
mean	5000.50000	1.569094e+07	NaN	650.528800	NaN	NaN	38.921800
std	2886.89568	7.193619e+04	NaN	96.653299	NaN	NaN	10.487806
min	1.00000	1.556570e+07	NaN	350.000000	NaN	NaN	18.000000
25%	2500.75000	1.562853e+07	NaN	584.000000	NaN	NaN	32.000000
50%	5000.50000	1.569074e+07	NaN	652.000000	NaN	NaN	37.000000
75%	7500.25000	1.575323e+07	NaN	718.000000	NaN	NaN	44.000000
max	10000.00000	1.581569e+07	NaN	850.000000	NaN	NaN	92.000000

In [13]:
```
data.columns
```

Out[13]:
```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

In [14]:
```
data = data.drop(['RowNumber', 'CustomerId', 'Surname'],axis=1)
```

In [15]:
```
data.head()
```

Out[15]:

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActi
0	619	France	Female	42	2	0.00	1	1	
1	608	Spain	Female	41	1	83807.86	1	0	
2	502	France	Female	42	8	159660.80	3	1	
3	699	France	Female	39	1	0.00	2	0	
4	850	Spain	Female	43	2	125510.82	1	1	

In [16]:
```
data['Geography'].unique()
```

Out[16]:
```
array(['France', 'Spain', 'Germany'], dtype=object)
```

In [17]:
```
data = pd.get_dummies(data,drop_first=True)
```

In [18]:
```
data.head()
```

Out[18]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedS
0	619	42	2	0.00	1	1	1	1013
1	608	41	1	83807.86	1	0	1	1125
2	502	42	8	159660.80	3	1	0	1139
3	699	39	1	0.00	2	0	0	938
4	850	43	2	125510.82	1	1	1	790

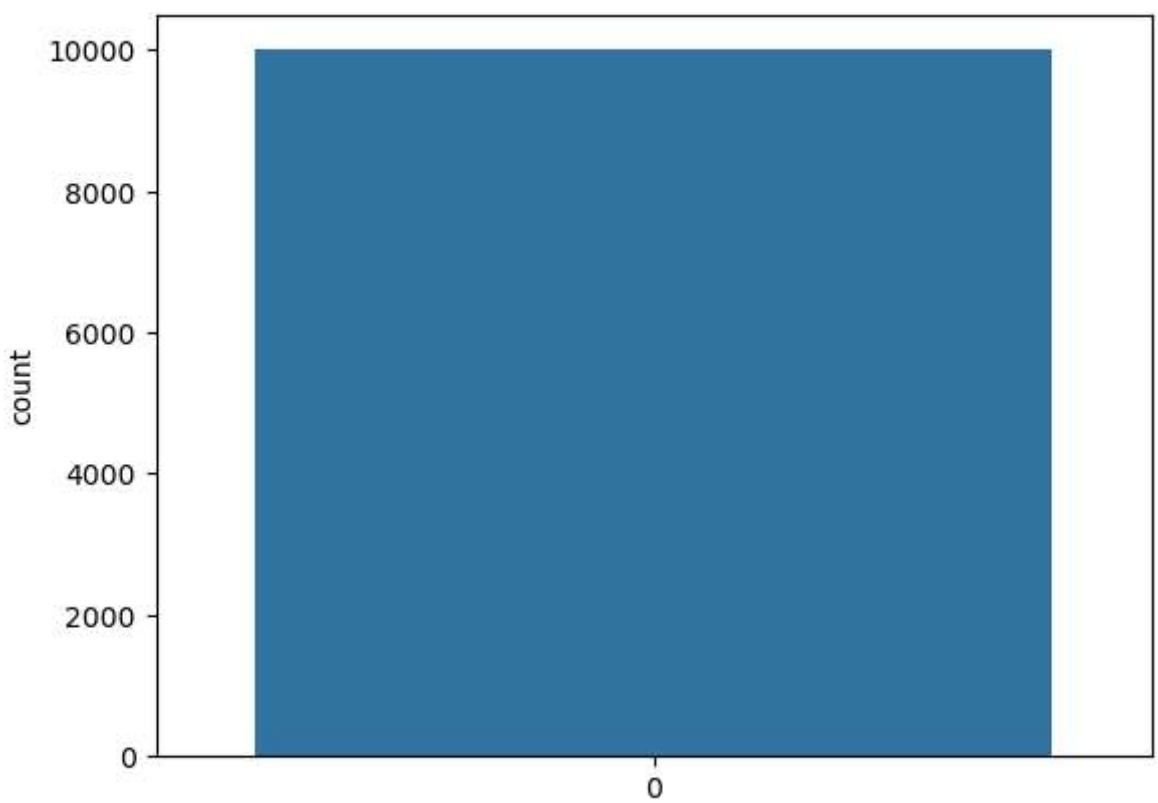
In [19]:
```
data['Exited'].value_counts()
```

Out[19]:
```
0    7963
1    2037
Name: Exited, dtype: int64
```

In [20]:
```
import seaborn as sns
```

In [21]:
```
sns.countplot(data['Exited'])
```

Out[21]:
```
<Axes: ylabel='count'>
```



In [22]:
```
X = data.drop('Exited',axis=1)
y = data['Exited']
```

In [23]:
```
from imblearn.over_sampling import SMOTE
```

In [24]:
```
X_res,y_res = SMOTE().fit_resample(X,y)
```

In [25]:
```
y_res.value_counts()
```

Out[25]:
```
1    7963
0    7963
Name: Exited, dtype: int64
```

In [26]:
```
from sklearn.model_selection import train_test_split
```

In [27]:
```
X_train,X_test,y_train,y_test=train_test_split(X_res,y_res,test_size=0.20,

```

In [28]:
```
from sklearn.preprocessing import StandardScaler

```

In [29]:
```
sc= StandardScaler()

```

In [30]:
```
X_train=sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [31]:
```
X_train
```

Out[31]:
```
array([[-2.63262343, -0.90308021,  0.07218677, ..., -0.56731914,
         2.16682427, -0.84920931],
       [ 1.09404851,  0.10035975, -1.40221723, ..., -0.56731914,
        -0.46150489,  1.17756599],
       [ 0.18962228, -0.10032824,  0.07218677, ...,  1.7626763 ,
        -0.46150489, -0.84920931],
       ...,
       [ 0.20051898,  0.20070374,  1.17798977, ..., -0.56731914,
        -0.46150489,  1.17756599],
       [-2.34930918,  0.80276772, -0.66501523, ...,  1.7626763 ,
        -0.46150489, -0.84920931],
       [ 0.52742003,  1.00345571, -1.40221723, ...,  1.7626763 ,
        -0.46150489,  1.17756599]])
```

Logistic Regression

In [32]:
```
from sklearn.linear_model import LogisticRegression
```

In [33]:
```
log = LogisticRegression()
```

In [34]:
```
log.fit(X_train,y_train)
```

Out[34]:
```
LogisticRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [35]:
```
y_pred1 = log.predict(X_test)
```

In [36]:
```
from sklearn.metrics import accuracy_score
```

In [39]:
```
accuracy_score(y_test,y_pred1)
```

Out[39]:
```
0.7818581293157565
```

In [40]:
```

accuracy_score(y_test,y_pred1)
```

Out[40]:
```
0.7818581293157565
```

In [41]:
```
from sklearn.metrics import precision_score,recall_score,f1_score

```

In [42]:
```
precision_score(y_test,y_pred1)

```

Out[42]:
```
0.7701511335012594
```

In [43]:
```
recall_score(y_test,y_pred1)

```

Out[43]:
```
0.7875080489375402
```

In [44]:
```
f1_score(y_test,y_pred1)

```

Out[44]:
```
0.778732887615409
```

SVC

In [52]:
```
from sklearn import svm
```

In [53]:
```
svm = svm.SVC()

```

In [54]:
```
svm.fit(X_train,y_train)

```

Out[54]:
```
SVC()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [55]:
```
y_pred2 = svm.predict(X_test)

```

In [56]:
```
accuracy_score(y_test,y_pred2)

```

Out[56]:
```
0.842435655994978
```

In [57]:
```
precision_score(y_test,y_pred2)

```

Out[57]:
```
0.8311279143037177
```

KNeighbors Classifier

In [58]:
```
from sklearn.neighbors import KNeighborsClassifier

```

In [59]:
```
knn = KNeighborsClassifier()

```

In [60]:
```
knn.fit(X_train,y_train)

```

Out[60]:
```
KNeighborsClassifier()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [61]:
```
y_pred3 = knn.predict(X_test)

```

In [62]:
```
accuracy_score(y_test,y_pred3)
```

Out[62]: 0.8232893910860013

In [63]:
```
precision_score(y_test,y_pred3)
```

Out[63]: 0.8025672371638142

Decision Tree Classifier

In [64]:
```
from sklearn.tree import DecisionTreeClassifier
```

In [65]:
```
dt = DecisionTreeClassifier()
```

In [66]:
```
dt.fit(X_train,y_train)
```

Out[66]: DecisionTreeClassifier()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [67]:
```
y_pred4 = dt.predict(X_test)
```

In [68]:
```
accuracy_score(y_test,y_pred4)
```

Out[68]: 0.8088512241054614

In [69]:
```
precision_score(y_test,y_pred4)
```

Out[69]: 0.7881562881562881

Random Forest Classifier

In [70]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [71]:
```
rf = RandomForestClassifier()

```

In [72]:
```
rf.fit(X_train,y_train)

```

Out[72]:
```
RandomForestClassifier()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [73]:
```
y_pred5 = rf.predict(X_test)

```

In [74]:
```
accuracy_score(y_test,y_pred5)

```

Out[74]:
```
0.8634651600753296
```

In [75]:
```
precision_score(y_test,y_pred5)

```

Out[75]:
```
0.8489388264669163
```

Gradient Boosting Classifier

In [80]:
```
from sklearn.ensemble import GradientBoostingClassifier

```

In [81]:
```
gbc = GradientBoostingClassifier()

```

In [82]:
```
gbc.fit(X_train,y_train)

```

Out[82]:
```
GradientBoostingClassifier()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [83]:
```
y_pred6 = gbc.predict(X_test)

```

In [84]:
```
accuracy_score(y_test,y_pred6)

```

Out[84]: 0.8471437539234149

In [85]:
```
precision_score(y_test,y_pred6)

```

Out[85]: 0.8412291933418694

In [86]:
```
final_data=pd.DataFrame({'Models':['LR','SVC','KNN','DT','RF','GBC'],
                         'ACC':[accuracy_score(y_test,y_pred1),
                                accuracy_score(y_test,y_pred2),
                                accuracy_score(y_test,y_pred3),
                                accuracy_score(y_test,y_pred4),
                                accuracy_score(y_test,y_pred5),
                                accuracy_score(y_test,y_pred6)]})

```

In [87]:
```
final_data

```

Out[87]:

	Models	ACC
0	LR	0.781858
1	SVC	0.842436
2	KNN	0.823289
3	DT	0.808851
4	RF	0.863465
5	GBC	0.847144

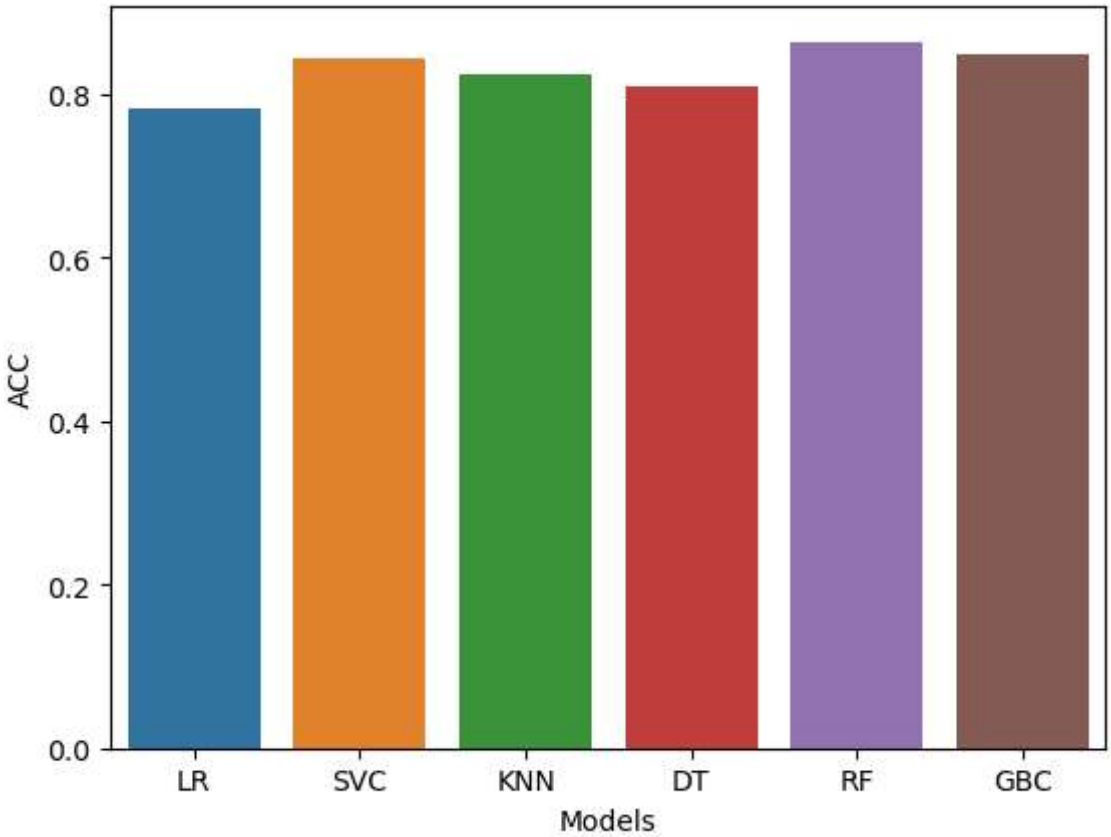
In [88]:
```
import seaborn as sns

```

In [90]:
```
sns.barplot(x=final_data['Models'], y=final_data['ACC'])

```

Out[90]: <Axes: xlabel='Models', ylabel='ACC'>



In [91]:
```
final_data=pd.DataFrame({'Models':['LR','SVC','KNN','DT','RF','GBC'],
                         'PRE':[precision_score(y_test,y_pred1),
                                precision_score(y_test,y_pred2),
                                precision_score(y_test,y_pred3),
                                precision_score(y_test,y_pred4),
                                precision_score(y_test,y_pred5),
                                precision_score(y_test,y_pred6)]})
```

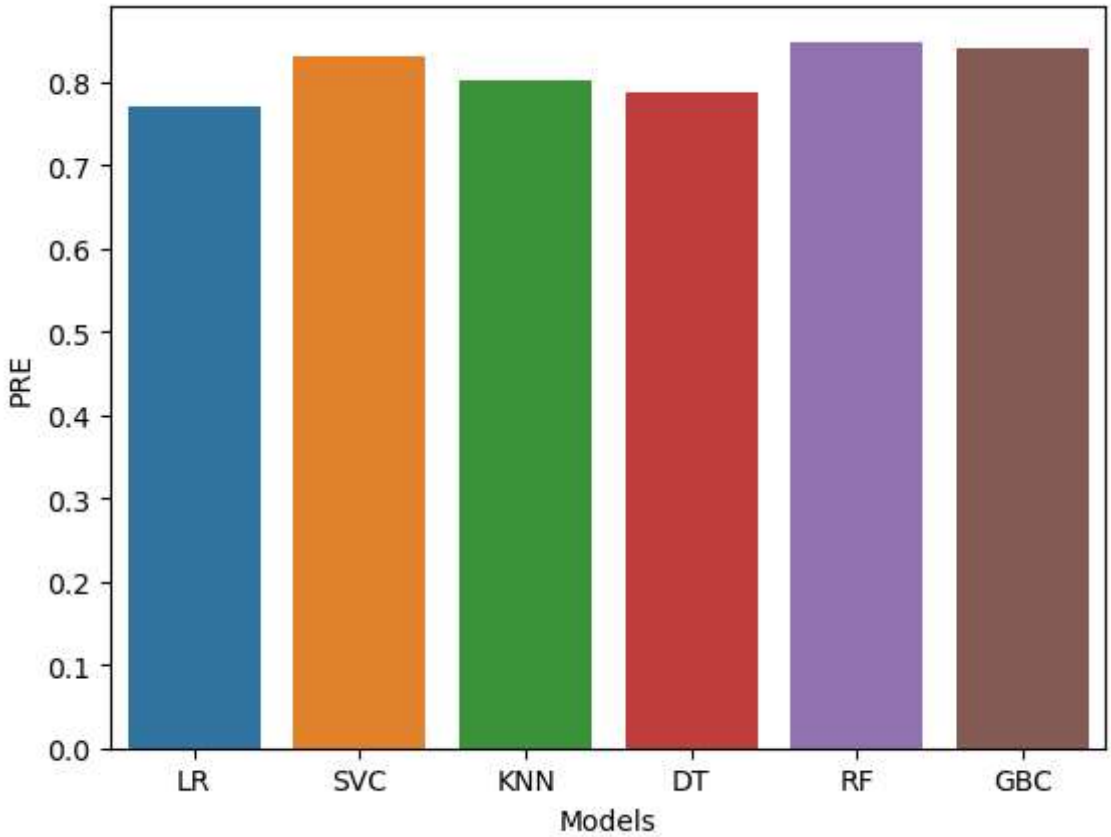
In [92]:
```
final_data
```

Out[92]:

	Models	PRE
0	LR	0.770151
1	SVC	0.831128
2	KNN	0.802567
3	DT	0.788156
4	RF	0.848939
5	GBC	0.841229

In [96]:
```
sns.barplot(x='Models', y='PRE', data=final_data)
```

Out[96]: <Axes: xlabel='Models', ylabel='PRE'>



In [97]:
```
X_res=sc.fit_transform(X_res)
```

In [98]:
```
rf.fit(X_res,y_res)

```

Out[98]: RandomForestClassifier()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [99]:
```
import joblib

```

In [100]:
```
joblib.dump(rf,'churn_predict_model')

```

Out[100]: ['churn_predict_model']

In [101]:
```
model = joblib.load('churn_predict_model')

```

In [102]:
```
data.columns

```

Out[102]:
```
Index(['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCar
d',
       'IsActiveMember', 'EstimatedSalary', 'Exited', 'Geography_Germany',
       'Geography_Spain', 'Gender_Male'],
      dtype='object')
```

In [103]:
```
model.predict([[619,42,2,0.0,0,0,0,101348.88,0,0,0]])

```

Out[103]:
```
array([1], dtype=int64)
```

APP CODE

In [3]:
```
from tkinter import Tk, Label, Entry, Button, mainloop
import joblib

```

In [5]:

```python
def show_entry_fields():
    p1=int(e1.get())
    p2=int(e2.get())
    p3=int(e3.get())
    p4=float(e4.get())
    p5=int(e5.get())
    p6=int(e6.get())
    p7=int(e7.get())
    p8=float(e8.get())
    p9=int(e9.get())
    if p9 == 1:
        Geography_Germany=1
        Geography_Spain=0
        Geography_France=0
    elif p9 == 2:
        Geography_Germany=0
        Geography_Spain=1
        Geography_France=0
    elif p9 == 3:
        Geography_Germany=0
        Geography_Spain=0
        Geography_France=1
    p10=int(e10.get())
    model = joblib.load('churn_model')
    result=model.predict(sc.transform([[p1,p2,p3,p4,
                            p5,p6,
                            p7,p8,Geography_Germany,Geography_Spain,p10]]))

    if result == 0:
        Label(master, text="No Exit").grid(row=31)
    else:
        Label(master, text="Exit").grid(row=31)


master = Tk()
master.title("Bank Customers Churn Prediction Using Machine Learning")


label = Label(master, text = "Customers Churn Prediction Using ML"
                            , bg = "black", fg = "white"). \
                                grid(row=0,columnspan=2)


Label(master, text="CreditScore").grid(row=1)
Label(master, text="Age").grid(row=2)
Label(master, text="Tenure").grid(row=3)
Label(master, text="Balance").grid(row=4)
Label(master, text="NumOfProducts").grid(row=5)
Label(master, text="HasCrCard").grid(row=6)
Label(master, text="IsActiveMember").grid(row=7)
Label(master, text="EstimatedSalary").grid(row=8)
Label(master, text="Geography").grid(row=9)
Label(master,text="Gender").grid(row=10)


e1 = Entry(master)
e2 = Entry(master)
```

```
e3 = Entry(master)
e4 = Entry(master)
e5 = Entry(master)
e6 = Entry(master)
e7 = Entry(master)
e8 = Entry(master)
e9 = Entry(master)
e10 = Entry(master)


e1.grid(row=1, column=1)
e2.grid(row=2, column=1)
e3.grid(row=3, column=1)
e4.grid(row=4, column=1)
e5.grid(row=5, column=1)
e6.grid(row=6, column=1)
e7.grid(row=7, column=1)
e8.grid(row=8, column=1)
e9.grid(row=9, column=1)
e10.grid(row=10,column=1)

Button(master, text='Predict', command=show_entry_fields).grid()

mainloop()

```

In [ ]:

```

```